EXPERIMENT - 9

AIM:

Implement Recurrent Neural Network (RNN) / LSTM for time series or text data

1. Dataset Source

Dataset Name: Air Quality Data Set (UCI Machine Learning Repository)
File Used: Air Quality Data Set

2. Dataset Description

The dataset contains 9,358 instances of hourly averaged responses from an array of 5 metal oxide chemical sensors embedded in an Air Quality Chemical Multisensor Device. It is used to predict air pollutant concentrations.

- **Type:** Multivariate Time Series (Numerical)
- **Domain:** Environmental Monitoring / Smart Cities
- **Target:** Regression (Predicting Benzene concentration: C6H6(GT))
- **Key Features:**

Feature	Description	Data Type
CO(GT)	True hourly averaged concentration of Carbon Monoxide	Numerical
PT08.S1(CO)	Tin oxide hourly averaged sensor response	Numerical
C6H6(GT)	True hourly averaged Benzene concentration (Target)	Numerical
T	Temperature in °C	Numerical
RH	Relative Humidity (%)	Numerical

3.Mathematical Formulation of the Algorithm

A. Recurrent Neural Network (RNN) / LSTM Standard ANNs assume all inputs are independent. RNNs are designed for sequential data as they possess "memory" by feeding the output of a previous step back into the current step.

B. Long Short-Term Memory (LSTM) LSTMs are a special kind of RNN capable of learning long-term dependencies. They solve the "Vanishing Gradient" problem using a **Gating Mechanism**:

- **Forget Gate:** Decides what information from the previous cell state to discard.
- **Input Gate:** Decides which new information to store in the cell state.
- **Output Gate:** Decides what part of the cell state to output as the hidden state.

C. Optimization and Loss:

- **Mean Squared Error (MSE):** Since this is a regression task, we minimize the average squared difference between predicted and actual values.
- **Adam Optimizer:** Adjusts the learning rate dynamically for faster convergence in time-series data.

4. Algorithm Limitations

- **Sequential Requirement:** LSTMs are harder to parallelize than CNNs because each time step depends on the previous one.
- **Sensitivity to Scaling:** LSTMs are highly sensitive to the scale of input data; features with different ranges can cause the model to fail during training.
- **Complexity:** They require careful tuning of "look-back" windows and hidden units to avoid overfitting on seasonal trends.

5. Methodology / Workflow

1. **Load Dataset:** Import the Air Quality CSV and handle missing values (often marked as -200 in this specific dataset).
2. **Data Preprocessing: * Normalization:** Apply Z-score standardization so that different chemical concentrations are on the same scale.

    - **Time Series Formatting:** Reshape data into sequences (samples, time steps, features).

3. **Building the Architecture:** Create a model with an **LSTM Layer** (for temporal memory), **Dropout** (to prevent overfitting), and a **Dense Layer** (for final regression output).
4. **Model Compilation:** Use 'Adam' optimizer and 'mean_squared_error' (MSE) as the loss function.
5. **Training:** Fit the model on the chronological training split (80% training, 20% testing).
6. **Performance Analysis:** Evaluate using **Root Mean Squared Error (RMSE)**, Time Series plots (Actual vs. Predicted), and Regression Correlation plots.

6. Performance Analysis

- **RMSE (Root Mean Squared Error):** This metric provides the average magnitude of error in the same units as the Benzene concentration. A lower RMSE indicates a more accurate "pollution forecaster."
- **Time Series Overlay:** By plotting the actual sensor data against the LSTM predictions, we observe how well the model captures "peaks" (high pollution hours) and "troughs" (cleaner hours).
- **Error Distribution:** Since this is regression, we analyze the Residual Histogram. A peak at zero error confirms that the model has no systematic bias in its predictions.

7. Hyperparameter Tuning

- **Hidden Units:** We used 100 hidden units in the LSTM layer to capture the complex relationship between temperature, humidity, and chemical reactions.
- **Learning Rate Schedule:** A piecewise learning rate was used to slow down updates as the model approached the global minimum, ensuring stability.
- **Gradient Threshold:** Set to 1 to prevent "exploding gradients," a common issue when training recurrent architectures on long sequences.

Code:

1. **Load and Explore Data**

```
opts = detectImportOptions('AirQualityUCI.xlsx');

opts.VariableNamingRule = 'preserve';

data = readtable('AirQualityUCI.xlsx', opts);


% Remove empty rows/columns

data = rmmissing(data);


% Select Features: CO(GT), PT08.S1, C6H6(GT), T, RH

selectedData = data{:, {'CO(GT)', 'PT08.S1(CO)', 'C6H6(GT)', 'T', 'RH'}};


% Filter out -200 values

selectedData(selectedData == -200) = NaN;

selectedData = fillmissing(selectedData, 'linear');
```

2. Data Preprocessing for Time Series

```
% Target variable is C6H6 (Column 3)
targetData = selectedData(:, 3)';
features = selectedData';
% Split into Training (80%) and Testing (20%)
numTimeSteps = size(features, 2);
numTrain = floor(0.8 * numTimeSteps);

trainFeatures = features(:, 1:numTrain);
testFeatures = features(:, numTrain+1:end);
trainTarget = targetData(1:numTrain);
testTarget = targetData(numTrain+1:end);

% Standardize the data (Z-score normalization)
mu = mean(trainFeatures, 2);
sig = std(trainFeatures, 0, 2);
trainFeaturesNorm = (trainFeatures - mu) ./ sig;
testFeaturesNorm = (testFeatures - mu) ./ sig;

% Standardize targets
muT = mean(trainTarget);
sigT = std(trainTarget);
trainTargetNorm = (trainTarget - muT) ./ sigT;
```

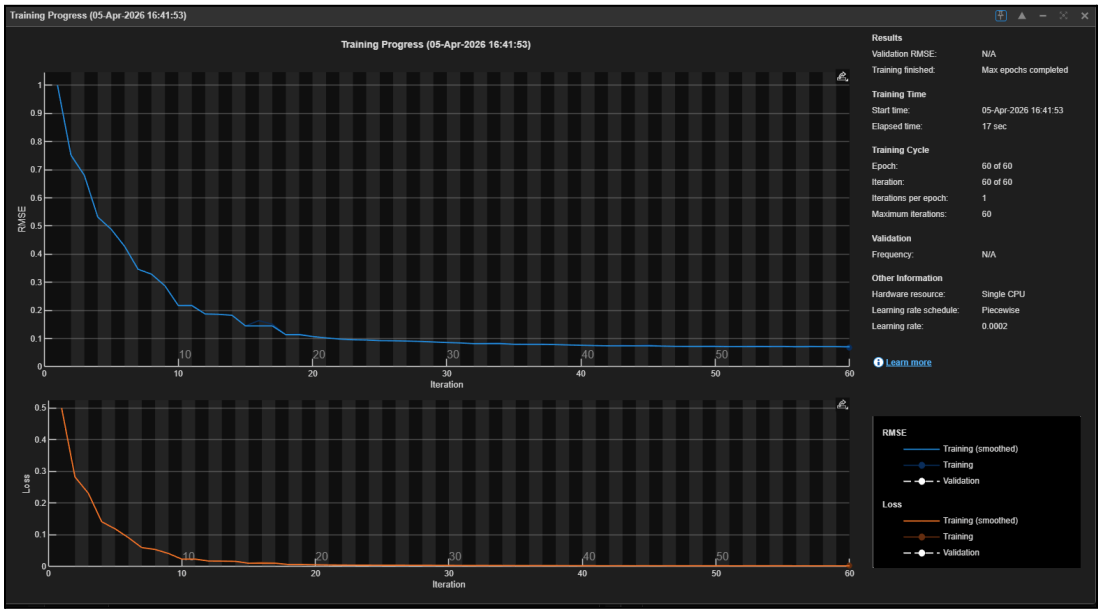
3. **Define LSTM Architecture**

```
numInputs = size(features, 1);
numResponses = 1;
numHiddenUnits = 100;

layers = [ ...
    sequenceInputLayer(numInputs)
    lstmLayer(numHiddenUnits, 'OutputMode', 'sequence')
    fullyConnectedLayer(numHiddenUnits)
    dropoutLayer(0.2)
    fullyConnectedLayer(numResponses)
    regressionLayer];
```

4. **Train the LSTM Network**

```
options = trainingOptions('adam', ...
    'MaxEpochs', 60, ...
    'GradientThreshold', 1, ...
    'InitialLearnRate', 0.005, ...
    'LearnRateSchedule', 'piecewise', ...
    'LearnRateDropPeriod', 20, ...
    'LearnRateDropFactor', 0.2, ...
    'Verbose', 0, ...
    'Plots', 'training-progress');
```

net = trainNetwork(trainFeaturesNorm, trainTargetNorm, layers, options);



5. Forecasting and Performance Analysis

% Predict on test data

YPredNorm = predict(net, testFeaturesNorm);

% Unstandardize predictions to get actual Benzene levels

YPred = (YPredNorm * sigT) + muT;

figure;

subplot(2,1,1);

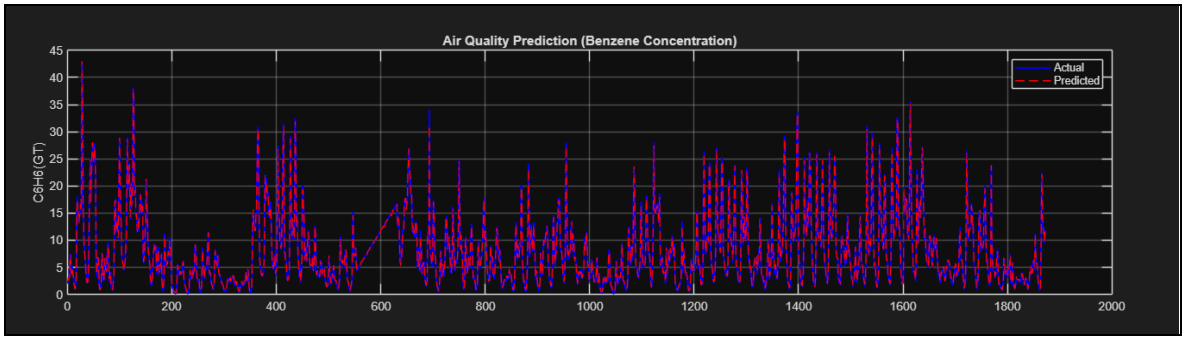
plot(testTarget, 'b', 'LineWidth', 1.5); hold on;

plot(YPred, 'r--', 'LineWidth', 1);

legend('Actual', 'Predicted');

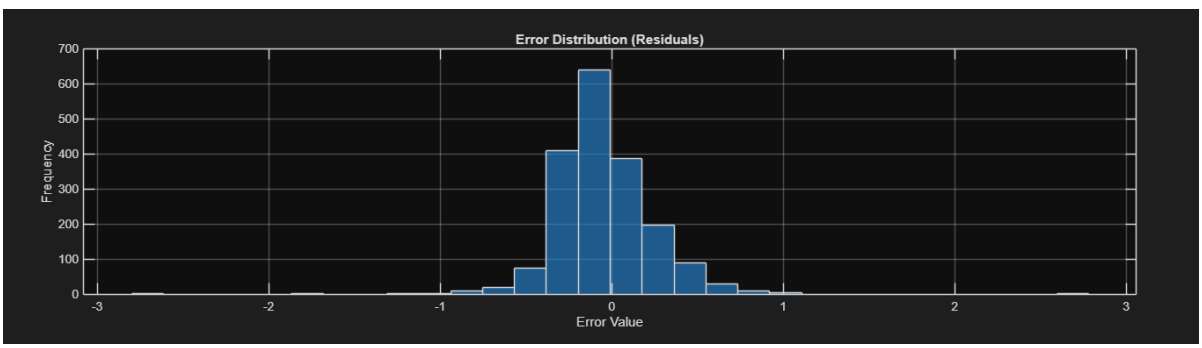
title('Air Quality Prediction (Benzene Concentration)');

ylabel('C6H6(GT)'); grid on;



6. Error Histogram

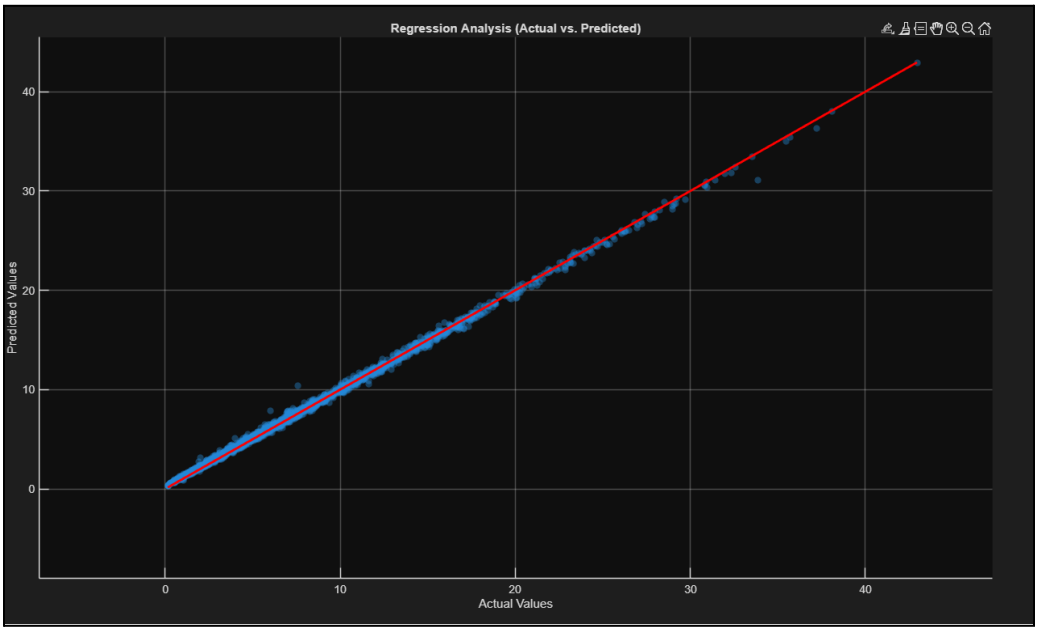
```
errors = testTarget - YPred;

subplot(2,1,2);

histogram(errors, 30);

title('Error Distribution (Residuals)');

xlabel('Error Value'); ylabel('Frequency'); grid on;
```



7. Regression Plot

```
figure;

scatter(testTarget, YPred, 'filled', 'MarkerFaceAlpha', 0.4);

hold on;

plot([min(testTarget) max(testTarget)], [min(testTarget) max(testTarget)], 'r', 'LineWidth',
2);

xlabel('Actual Values'); ylabel('Predicted Values');

title('Regression Analysis (Actual vs. Predicted)');
```

grid on;



CONCLUSION

A Long Short-Term Memory (LSTM) network was successfully implemented to forecast urban air quality. Unlike standard feed-forward networks, the LSTM's ability to maintain a "cell state" allowed it to learn the temporal trends of chemical concentrations over time. The results, visualized through regression plots and time-series overlays, demonstrate that LSTM architectures are highly effective for environmental monitoring and other time-sensitive predictive tasks where historical context is crucial for future accuracy.